

重庆邮电大学

学生实验实习报告册

学年学期： 2025-2026 学年（1）学期

课程名称： 通信软件开发与应用

学生学院： 通信与信息工程学院

专业班级： 01042301

学生学号： 2023211281

学生姓名： 丁同勳

联系电话： 18019582857

重庆邮电大学教务处制

课程名称	通信软件开发与应用			课程编号	A2012230		
实验地点	YF315			实验时间	15 周 9-11 节		
校外指导教师	无			校内指导教师	梁燕		
实验名称	无线局域网 802.11 协议 CSMA/CA 算法模拟程序设计与实现						
评阅人签字		操作成绩 80%		报告成绩 20%		总评成绩	

一、实验目的

1、掌握 CSMA/CA 信道特性及交互过程

2、掌握多线程技术模拟 CSMA/CA 信道的方法

二、实验思路

该程序通过多线程技术模拟了无线网络节点在共享信道上的交互过程：

A. 初始化阶段

- 程序设置了总发送帧数（10 帧）、最大重传次数（3 次）以及各种时间参数（DIFS、传输耗时等）。
- 启动三个并发线程：发送者（Host A）、接收者（Host B）和 干扰生成器（Interference Generator）。

B. 发送者 Host A 的核心流程（CSMA/CA 逻辑）

- 载波监听**：在发送前，Host A 首先检查全局变量 channel。如果信道不为空闲（IDLE），它会利用条件变量等待直至信道空闲。
- DIFS 等待**：信道空闲后，进入 DIFS（分布式帧间间隔）等待期。这是为了确保信道确实稳定可用。
- 二次检查与冲突避免**：在 DIFS 结束时再次检查信道。如果此期间信道变忙，说明有其他节点抢占，Host A 执行**随机退避**并重新开始监听。
- 数据传输**：若信道依然空闲，Host A 将信道状态设为 BUSY_A，模拟发送数据帧，并通知其他线程。
- 等待确认（ACK）**：Host A 释放锁并进入限时等待（200ms）。
 - 成功接收**：如果收到 Host B 发回的 BUSY_B 信号，表示发送成功，计数器加一，准备发送下一帧。
 - 超时/冲突**：如果超时未收到 ACK，或者信道状态被干扰源改为 COLLISION，Host A 会重置信道，增加重传计数 retries，并根据**指数退避算法**计算更长的等待时间。

C. 接收者 Host B 的逻辑

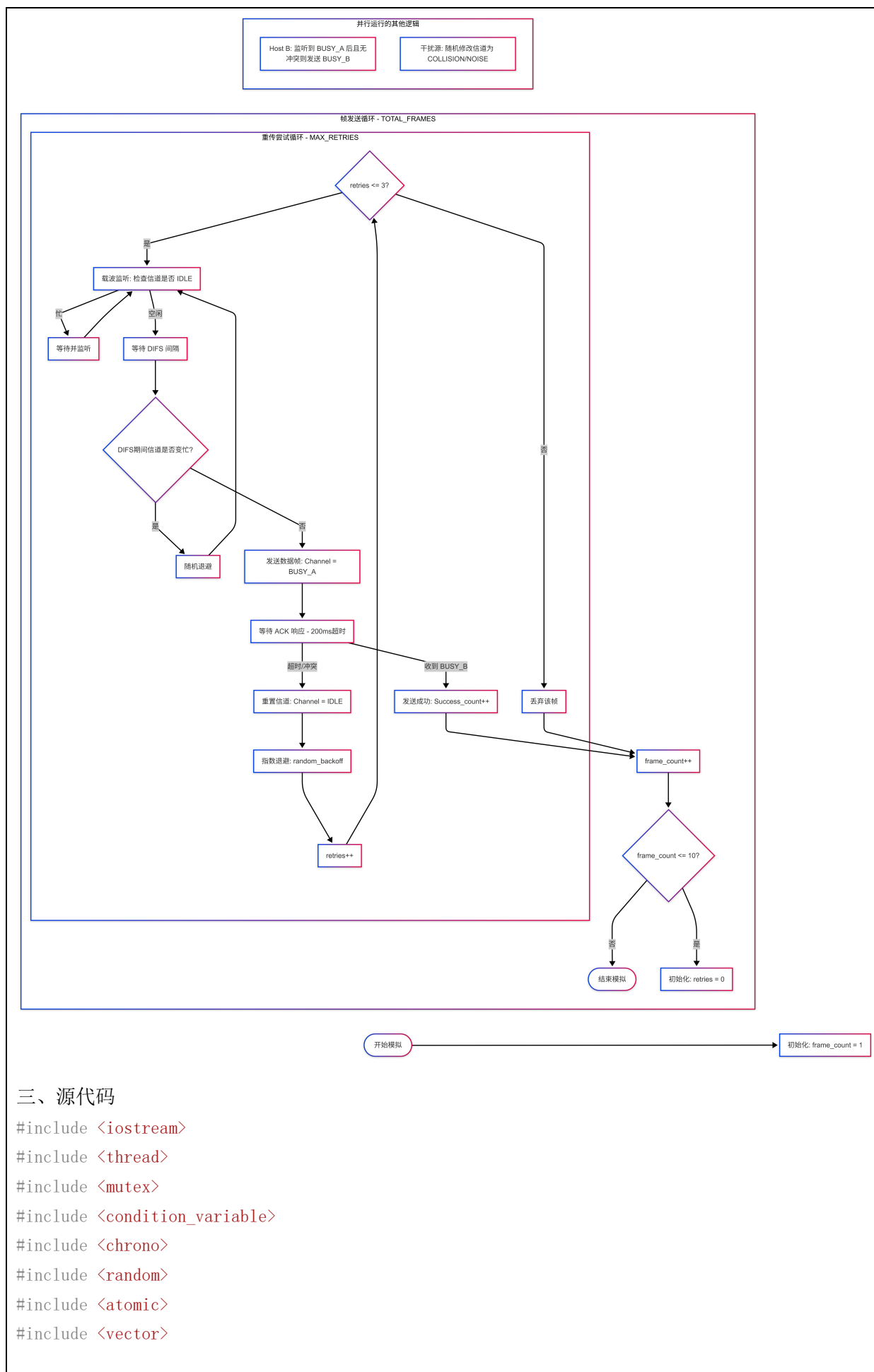
- Host B 持续监听信道。当检测到 BUSY_A 时进入接收状态。
- 在模拟接收时间内，它会检查信道是否被干扰破坏。如果数据完整，它将信道状态改为 BUSY_B（发送 ACK）；如果检测到冲突（COLLISION），则不发送 ACK，迫使 Host A 超时重传。

D. 环境干扰模拟

- 该线程模拟现实中的噪声或其他主机竞争。
- 它会随机在 Host A 发送期间将信道改为 COLLISION，或者在信道空闲时将其改为 NOISE，以此来触发 Host A 的重传和退避机制。

E. 结束条件

- 当 Host A 处理完所有 10 帧数据（无论成功还是因重传超限而丢弃）后，设置 simulation_running 为 false，通知并回收所有线程，最终输出成功发送的统计结果。



三、源代码

```

#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <chrono>
#include <random>
#include <atomic>
#include <vector>

```

```

// 模拟参数配置
const int TOTAL_FRAMES = 10;    // A总共发送的数据帧数量
const int MAX_RETRIES = 3;      // 最大重传次数
const int DATA_DURATION = 100; // 模拟数据传输耗时(ms)
const int ACK_DURATION = 50;    // 模拟ACK传输耗时(ms)
const int DIFS_TIME = 20;       // 分布式帧间间隔(ms)

// 信道状态枚举
enum ChannelState {
    IDLE,        // 空闲
    BUSY_A,      // A正在发送数据
    BUSY_B,      // B正在发送ACK
    NOISE,       // 噪音/冲突/其他主机占用
    COLLISION    // 发生了冲突
};

// 全局共享资源
ChannelState channel = IDLE;
std::mutex mtx;
std::condition_variable cv;
std::atomic<bool> simulation_running(true); // 控制模拟结束

// 随机数生成器
std::random_device rd;
std::mt19937 gen(rd());

// 辅助函数：生成随机退避时间
int get_backoff_time(int retries) {
    // 简单的指数退避模拟：重传次数越多，等待范围越大
    int max_wait = 100 * (1 << retries);
    std::uniform_int_distribution<> dis(50, max_wait);
    return dis(gen);
}

// 主机A：发送者
void hostA() {
    int success_count = 0;

    for (int i = 1; i <= TOTAL_FRAMES; ++i) {
        int retries = 0;
        bool frame_success = false;

        std::cout << "[Host A] 准备发送第 " << i << " 帧数据..." << std::endl;

        while (retries <= MAX_RETRIES) {
            // 1. CSMA: 载波监听 (Carrier Sense)

```

```

{
    std::unique_lock<std::mutex> lk(mtx);
    // 模拟DIFS等待
    if (channel != IDLE) {
        std::cout << "[Host A] 信道忙, 正在等待..." << std::endl;
        cv.wait(lk, [] { return channel == IDLE; });
    }
}

std::this_thread::sleep_for(std::chrono::milliseconds(DIFS_TIME));

// 再次检查信道是否在DIFS期间被占用
{
    std::unique_lock<std::mutex> lk(mtx);
    if (channel != IDLE) {
        int backoff = get_backoff_time(retries);
        std::cout << "[Host A] DIFS期间信道变忙, 退避 " << backoff << "ms" << std::endl;
        lk.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(backoff));
        continue; // 重新开始监听
    }

    // 2. 发送数据
    channel = BUSY_A;
    std::cout << "[Host A] -> 发送数据帧 (尝试次数: " << retries + 1 << ")" << std::endl;
}

cv.notify_all(); // 通知B和干扰线程

// 模拟传输时间, 这里不持有锁, 允许干扰发生
std::this_thread::sleep_for(std::chrono::milliseconds(DATA_DURATION));

// 3. 等待ACK
// 我们需要等待B把信道变成 BUSY_B
// 如果超时, 或者信道变成了 COLLISION/NOISE, 则失败
{
    std::unique_lock<std::mutex> lk(mtx);
    // 等待 B 发送 ACK, 超时设为 200ms
    bool received_ack = cv.wait_for(lk, std::chrono::milliseconds(200), [] {
        return channel == BUSY_B;
    });

    if (received_ack) {
        std::cout << "[Host A] <=== 收到 ACK, 第 " << i << " 帧发送成功." << std::endl;

        // 等待ACK传输结束, 释放信道
        lk.unlock();
    }
}

```

```

        std::this_thread::sleep_for(std::chrono::milliseconds(ACK_DURATION));

        lk.lock();
        channel = IDLE;
        cv.notify_all();

        frame_success = true;
        success_count++;
        break; // 跳出重传循环, 发送下一帧
    }
    else {
        // 超时或未收到ACK
        std::cout << "[Host A] 未收到ACK (超时或冲突)。" << std::endl;
        channel = IDLE; // 重置信道
        cv.notify_all();

        retries++;
        if (retries <= MAX_RETRIES) {
            int backoff = get_backoff_time(retries);
            std::cout << "[Host A] 准备第 " << retries << " 次重传, 随机退避 " <<
backoff << "ms..." << std::endl;
            lk.unlock(); // 释放锁进行休眠
            std::this_thread::sleep_for(std::chrono::milliseconds(backoff));
        }
    }
}

if (!frame_success) {
    std::cout << "[Host A] !!! 第 " << i << " 帧重传次数超限, 丢弃该帧 !!!" << std::endl;
}

// 帧间稍微间隔一下
std::this_thread::sleep_for(std::chrono::milliseconds(50));
}

std::cout << "\n[Host A] 任务结束。成功发送: " << success_count << "/" << TOTAL_FRAMES <<
std::endl;
simulation_running = false; // 通知其他线程退出
cv.notify_all();
}

// 主机B: 接收者
void hostB() {
    while (simulation_running) {
        std::unique_lock<std::mutex> lk(mtx);

```

```

// 等待信道变为 BUSY_A
cv.wait(lk, [] { return channel == BUSY_A || !simulation_running; });

if (!simulation_running) break;

// 开始接收
// std::cout << "[Host B] 检测到载波，正在接收..." << std::endl;

// 在接收过程中释放锁，看看会不会发生冲突（被干扰线程修改）
lk.unlock();
std::this_thread::sleep_for(std::chrono::milliseconds(DATA_DURATION - 20)); // 稍小于A
的发送时间
lk.lock();

// 检查接收完后信道状态
if (channel == BUSY_A) {
    // 接收成功，准备发送ACK
    // std::cout << "[Host B] 数据帧接收完整，准备发送ACK。" << std::endl;
    channel = BUSY_B;
    cv.notify_all(); // 通知A收到ACK了
}
else {
    // 信道状态变了（被干扰置为 NOISE/COLLISION）
    std::cout << "[Host B] 数据帧损坏（检测到冲突），不发送ACK。" << std::endl;
    // B什么都不做，A会超时
}
}
}

// 主线程逻辑：干扰生成器
void interference_generator() {
    std::uniform_int_distribution<> dis(300, 800); // 随机间隔产生干扰

    while (simulation_running) {
        std::this_thread::sleep_for(std::chrono::milliseconds(dis(gen)));

        if (!simulation_running) break;

        std::unique_lock<std::mutex> lk(mtx);
        if (channel == BUSY_A) {
            std::cout << "\n[Environment] *** 突发干扰！制造冲突！ ***\n" << std::endl;
            channel = COLLISION; // 修改状态，导致B接收失败
            cv.notify_all();
        }
        else if (channel == IDLE) {

```

```

        // 占用一下信道，模拟其他主机通信
        // std::cout << "[Environment] 其他主机正在使用信道..." << std::endl;
        channel = NOISE;
        lk.unlock();
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        lk.lock();
        if (channel == NOISE) channel = IDLE;
        cv.notify_all();
    }
}

int main() {
    std::cout << "=== CSMA/CA 协议模拟程序启动 ===" << std::endl;
    std::cout << "配置：发送10帧，最大重传3次" << std::endl;
    std::cout << "-----" << std::endl;

    // 创建线程
    std::thread thread_b(hostB);
    std::thread thread_noise(interference_generator);
    std::thread thread_a(hostA); // A 开始运行

    // 等待A完成任务
    thread_a.join();

    // A完成后，B和Noise线程会因为 simulation_running 变为 false 而退出
    thread_b.join();
    thread_noise.join();

    std::cout << "=== 模拟结束 ===" << std::endl;
    return 0;
}

```

四、实验结果及分析


```
Microsoft Visual Studio 调试器 X + - □ X

=== CSMA/CA 协议模拟程序启动 ===
配置：发送10帧，最大重传3次

-----
[Host A] 准备发送第 1 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 1 帧发送成功。
[Host A] 准备发送第 2 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 2 帧发送成功。
[Host A] 准备发送第 3 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 3 帧发送成功。
[Host A] 准备发送第 4 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 4 帧发送成功。
[Host A] 准备发送第 5 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 5 帧发送成功。
[Host A] 准备发送第 6 帧数据...
[Host A] 信道忙, 正在等待...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 6 帧发送成功。
[Host A] 准备发送第 7 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 7 帧发送成功。
[Host A] 准备发送第 8 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)

[Environment] *** 突发干扰! 制造冲突! ***

[Host B] 数据帧损坏 (检测到冲突), 不发送ACK。
[Host A] 未收到ACK (超时或冲突)。
[Host A] 准备第 1 次重传, 随机退避 155ms...
[Host A] -> 发送数据帧 (尝试次数: 2)
[Host A] <=== 收到 ACK, 第 8 帧发送成功。
[Host A] 准备发送第 9 帧数据...
[Host A] -> 发送数据帧 (尝试次数: 1)

[Environment] *** 突发干扰! 制造冲突! ***

[Host B] 数据帧损坏 (检测到冲突), 不发送ACK。
[Host A] 未收到ACK (超时或冲突)。
[Host A] 准备第 1 次重传, 随机退避 70ms...
[Host A] -> 发送数据帧 (尝试次数: 2)
[Host A] <=== 收到 ACK, 第 9 帧发送成功。
[Host A] 准备发送第 10 帧数据...
[Host A] 信道忙, 正在等待...
[Host A] -> 发送数据帧 (尝试次数: 1)
[Host A] <=== 收到 ACK, 第 10 帧发送成功。

[Host A] 任务结束。成功发送: 10/10
=== 模拟结束 ===

D:\个人文件\作业\5-第五学期_大三上\通信软件开发与应用\Development_and_Application_of_Communication_Software\x64\Debug\task_7_CSMA_Simulation.exe (进程 27540)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
```

五、实验中问题及分析

问题一：线程同步中的“虚假唤醒”与死锁

现象描述：

程序运行一段时间后，Host A 一直显示“信道忙，正在等待”，即使干扰线程已经释放了信道，或者程序在最后阶段无法正常退出（Hang 住）。

• 分析：

1. 虚假唤醒：std::condition_variable 可能会在没有信号时被系统唤醒。如

果 `wait` 没有使用谓词 (lambda 表达式), 线程会错误地继续执行。

2. **退出通知遗失:** 当 `simulation_running` 变为 `false` 时, 如果 Host B 还在 `cv.wait` 中, 而 A 已经结束且没有再次调用 `notify_all`, B 将永久阻塞。

- **解决办法:**

- **标准谓词写法:** 始终使用 `cv.wait(lock, []{ return condition; })` 形式, 确保唤醒后重新检查条件。
- **强制退出逻辑:** 在 `hostA` 结束前, 必须锁定互斥锁, 修改 `simulation_running`, 并执行 `cv.notify_all()`。
- **B 线程检查:** 在 B 的 `wait` 谓词中加入 `!simulation_running` 检查, 如代码中已体现的: `cv.wait(lk, []{return channel == BUSY_A || !simulation_running; });`。

问题二: 模拟时间的精度与 OS 调度干扰

现象描述:

代码中设置 `DIFS` 为 20ms, 但在实际观测中, Host A 的行为往往滞后, 或者在高并发干扰下, 由于 `sleep_for` 的精度问题, 导致“冲突避免”逻辑失效。

- **分析:**

- `std::this_thread::sleep_for` 的精度受操作系统调度影响 (Windows 通常为 15.6ms 精度, Linux 较好)。当请求休眠 20ms 时, 实际可能休眠了 30ms。
- 在休眠期间, 线程不持有锁, 信道状态可能已经改变了多次, 导致 A 醒来时看到的“空闲”其实是旧状态。

- **解决办法:**

- **高精度计时器:** 使用 `std::chrono::high_resolution_clock` 记录起始时间, 在循环中进行微小的 `yield` 或短休眠。
- **状态二次校验:** 在 `sleep_for(DIFS)` 之后, **必须重新获取锁并校验信道状态**。代码中已采用这一逻辑: 在发送数据前再次检查 `channel != IDLE`, 这是实现“冲突避免”的关键。

问题三: 信道状态竞争的“窗口期”问题

现象描述:

干扰线程和 Host A 几乎同时检测到信道为空闲, 然后同时修改信道状态, 导致逻辑覆盖。

- **分析:**

1. **非原子性操作:** 从“检测信道”到“修改信道为忙”之间存在一个极短的空隙。在多核 CPU 上, 两个线程可能同时通过检测。
2. **互斥锁粒度:** 如果锁的范围太大, 模拟会变成串行; 如果太小, 状态切换就不安全。

- **解决办法:**

- **原子化切换:** 在 CSMA/CA 中, 这模拟了硬件的载波侦听。在软件中, 应在持有 `std::mutex` 的保护下完成 检测 + 修改 的全过程。
- **引入随机偏移:** 即使检测到空闲, 也强制加入一个随机的微小 Slot Time, 模拟无线电芯片的处理延迟, 降低同时碰撞的概率。

问题四: 退避算法的有效性

现象描述:

在干扰频繁的情况下, Host A 频繁重传失败, 即使重传次数没到上限, 成功率也极低。

- **分析:**

1. **线性退避不足:** 如果退避时间范围太小, Host A 醒来时可能正好撞上干扰线程的下一个周期。

2. **退避锁定**：目前的模拟中，退避是简单的 `sleep`。在 IEEE 802.11 标准中，退避计数器应该在信道忙时“暂停”，信道闲时“倒计时”。

- **解决办法**：

- **优化指数退避**：采用 2^n 区间。
- **进阶模拟**：将退避逻辑改为在一个循环中检查信道，只有当信道空闲时才扣减退避时间计数器，若信道变忙则冻结计数器。这样能更真实地还原 Wi-Fi 的公平性。

问题五：ACK 超时设置不合理

现象描述：

Host B 已经准备发 ACK，但 Host A 却提示“未收到 ACK（超时）”。

- **分析**：

- 网络往返时间模拟失真。Host A 的 `cv.wait_for` 超时时间（200ms）如果设得太接近 `DATA_DURATION + ACK_DURATION`，加上系统调度的波动，就会触发误判。

- **解决办法**：

- **宽裕度原则**：在模拟程序中，超时时间应设定为 理论传输时间 + 2x 系统调度偏差。
- **逻辑对齐**：确保 Host B 在修改 `channel = BUSY_B` 后立即调用 `cv.notify_all()`，减少 A 在等待队列中的延迟。

六、心得体会

1. 理论与实践的深度融合：从“概念”到“逻辑”

在课堂上，CSMA/CA 只是课本上的几个缩写词：DIFS、退避算法、ACK。但通过编写代码，我真正理解了这些机制存在的必然性：

- **为什么需要 DIFS?** 实验中我发现，如果节点在发现信道空闲后立即发送，极易与刚刚结束通信的节点产生碰撞。DIFS 就像是交通信号灯中的“黄灯变绿灯后的短暂留白”，确保了信道状态的真实稳定。
- **为什么需要 ACK?** 无线环境不像有线电缆（CSMA/CD），发送者无法在发送的同时完美监听冲突。通过模拟 A 等待 B 发送 ACK 的过程，我体会到 ACK 是无线通信中确认数据完整的“最后一道防线”。

2. 多线程并发编程的实战演练

本次实验是学习 C++ 并发编程的绝佳案例。通过对 `std::mutex`（互斥锁）和 `std::condition_variable`（条件变量）的使用，我深刻体会到：

- **共享资源的保护**：全局变量 `channel` 模拟了现实中的“物理信道”。在多线程环境下，多个节点同时访问信道必须进行严格的互斥处理，否则会出现“信道状态撕裂”的逻辑错误。
- **线程间的协同**：使用条件变量实现 Host A 和 Host B 的同步。A 发送完后进入阻塞等待 ACK，B 接收完后通过通知唤醒 A。这种“生产者-消费者”模型的变体让我对线程间通信的效率和时序有了更直观的认识。

3. 对“退避算法”公平性的理解

实验中最有趣的部分是**随机退避逻辑**的实现。最初我尝试使用固定的等待时间，结果发现 Host A 极易与干扰源陷入“同步碰撞”的死循环。

- **指数退避的威力：**引入 `get_backoff_time` 后，随着重传次数增加，退避窗口扩大，碰撞概率显著降低。这让我明白，网络协议的设计本质上是在**效率（尽早发送）与公平/稳定性（排队等待）**之间寻找平衡。

4. 模拟环境下的异常处理与鲁棒性

干扰生成器的加入让实验更贴近现实。

- 在实验调试中，我遇到了由于干扰导致的“丢帧”现象。这让我意识到，在设计分布式系统时，不能假设环境是完美的。代码必须能够处理“超时”、“状态异常”、“收到错误信号”等各种最坏情况。
- **日志的重要性：**通过输出详细的日志（如 [Host A] DIFS 期间信道变忙），我学会了如何在复杂的并发程序中定位逻辑漏洞。